

INTERFACE

Voice dictation

Speak your prompts in the Claude Code CLI with hold-to-record or tap-to-record voice dictation.

Speak your prompts instead of typing them in the Claude Code CLI. Your speech is transcribed live into the prompt input, so you can mix voice and typing in the same message. Enable dictation with `/voice`, then either hold a key while you speak or tap once to start and again to send.

ⓘ Voice dictation requires Claude Code v2.1.69 or later. Tap mode requires v2.1.116 or later. Check your version with `claude --version`.

Dictation also works in [agent view](#). Hold or tap your push-to-talk key while the dispatch input or a peek-panel reply is focused to dictate to a background session.

Requirements

Voice dictation streams your recorded audio to Anthropic's servers for transcription. Audio is not processed locally. The speech-to-text service is only available when you authenticate with a Claude.ai account, and is not available when Claude Code is configured to use an Anthropic API key directly, Amazon Bedrock, Google Vertex AI, or Microsoft Foundry. Voice dictation is also not available when your organization has HIPAA compliance enabled. Transcription does not consume Claude messages or tokens and does not count toward the limits shown in `/usage`. See [data usage](#) for how Anthropic handles your data.

Voice dictation also needs local microphone access, so it does not work in remote environments such as [Claude Code on the web](#) or SSH sessions. In WSL, voice dictation requires WSLg for audio access. WSLg is included with WSL2 when installed from the Microsoft Store on Windows 10 or 11. If WSLg is not available, for example on WSL1, run Claude Code in native Windows instead.

Audio recording uses a built-in native module on macOS, Linux, and Windows. On Linux, if the native module cannot load, Claude Code falls back to `arecord` from ALSA utils or `rec` from SoX. If neither is available, `/voice` prints an install command for your package manager.

The Claude Code [VS Code extension](#) also supports voice dictation with the same Claude.ai account

requirement. It is not available in VS Code Remote sessions, including SSH, Dev Containers, and Codespaces, because the microphone is on your local machine and the extension runs on the remote host.

Enable voice dictation

Run `/voice` to enable dictation. The first time you enable it, Claude Code runs a microphone check. On macOS, this triggers the system microphone permission prompt for your terminal if it has never been granted.

```
/voice
Voice mode enabled (hold). Hold space to record. Dictation
language: en (/config to change).
```

`/voice` accepts an optional mode argument:

Command	Effect
<code>/voice</code>	Toggle on or off, keep the current mode
<code>/voice hold</code>	Enable in <u>hold mode</u>
<code>/voice tap</code>	Enable in <u>tap mode</u>
<code>/voice off</code>	Disable

Voice dictation persists across sessions. Set it directly in your user settings file instead of running `/voice` :

```
{
  "voice": {
    "enabled": true,
    "mode": "tap"
  }
}
```

While voice dictation is enabled, the input footer shows a `hold space to speak` hint when the prompt is empty. The hint reflects your current `voice:pushToTalk` binding and updates if you rebind the dictation key. The hint text is the same in both modes, and it does not appear if you have a

custom status line configured.

Transcription is tuned for coding vocabulary in both modes. Common development terms like `regex`, `OAuth`, `JSON`, and `localhost` are recognized correctly, and your current project name and git branch name are added as recognition hints automatically.

Hold to record

Hold mode is push-to-talk: recording runs while you hold the key and stops when you release it. This is the default mode.

Hold `Space` to start recording. Claude Code detects a held key by watching for rapid key-repeat events from your terminal, so there is a brief warmup before recording begins. The footer shows `keep holding...` during warmup, then switches to a live waveform once recording is active.

The first couple of key-repeat characters type into the input during warmup and are removed automatically when recording activates. A single `Space` tap still types a space, since hold detection only triggers on rapid repeat.

💡 To skip the warmup, switch to tap mode with `/voice tap`, or rebind to a modifier combination like `meta+k`. Modifier combos start recording on the first keypress.

Your speech appears in the prompt as you speak, dimmed until the transcript is finalized. Release `Space` to stop recording and finalize the text. The transcript is inserted at your cursor position and the cursor stays at the end of the inserted text, so you can mix typing and dictation in any order. Hold `Space` again to append another recording, or move the cursor first to insert speech elsewhere in the prompt:

```
> refactor the auth middleware to █  
  # hold space, speak "use the new token validation helper"  
> refactor the auth middleware to use the new token validation  
  helper█
```

By default, releasing the key inserts the transcript and waits for you to press `Enter`. Set `"autoSubmit": true` in the `voice` settings object to send the prompt automatically when you release the key, as long as the transcript is at least three words long.

Tap to record and send

Tap mode toggles recording with a single keypress: tap once to start, speak, then tap again to send the prompt. There is no warmup, and you don't need to keep the key held.

Enable tap mode with `/voice tap`. With the prompt input empty, tap `Space` to start recording. The footer shows a live waveform while recording. Tap `Space` again to stop. Claude Code inserts the transcript and submits the prompt automatically when the transcript is at least three words long. Shorter transcripts are inserted but not submitted, so an accidental tap does not send a stray word.

The first tap only starts recording when the prompt input is empty, so you can still type spaces normally while composing a message. The second tap stops recording regardless of input contents. Recording also stops automatically after 15 seconds of silence or two minutes total.

Change the dictation language

Voice dictation uses the same `language setting` that controls Claude's response language. If that setting is empty, dictation defaults to English. In the VS Code extension, if `language` is empty, dictation uses VS Code's `accessibility.voice.speechLanguage` setting before defaulting to English.

▸ Supported dictation languages

Set the language in `/config` or directly in settings. You can use either the BCP 47 language code or the language name:

```
{
  "language": "japanese"
}
```

If your `language` setting is not in the supported list, `/voice` warns you on enable and falls back to English for dictation. Claude's text responses are not affected by this fallback.

Rebind the dictation key

The dictation key is bound to `voice:pushToTalk` in the `Chat` context and defaults to `Space`. The same binding controls both hold and tap modes. Rebind it in `~/.claude/keybindings.json`:

```
{
  "bindings": [
    {
      "context": "Chat",
      "bindings": {
        "meta+k": "voice:pushToTalk",
        "space": null
      }
    }
  ]
}
```

The `voice:pushToTalk` action uses one key at a time. When you bind a custom key, it replaces the default `Space` binding rather than adding a second trigger, so the `"space": null` line in this example is for clarity and can be omitted without changing behavior.

In hold mode, avoid binding a bare letter key like `v` since hold detection relies on key-repeat and the letter types into the prompt during warmup. Use `Space`, or use a modifier combination like `meta+k` to start recording on the first keypress with no warmup. Tap mode has no warmup, so most keys work.

Some keys are not delivered to terminal applications and can't be bound at all. For example, `Caps Lock` shows an error if you try to bind it. See [customize keyboard shortcuts](#) for the full keybinding syntax and the list of reserved shortcuts.

Troubleshooting

Common issues when voice dictation does not activate or record:

- `Voice mode requires a Claude.ai account` : you are authenticated with an API key or a third-party provider. Run `/login` to sign in with a Claude.ai account.
- `Microphone access is denied` : grant microphone permission to your terminal in system settings. On macOS, go to System Settings → Privacy & Security → Microphone and enable your terminal app, then run `/voice` again. On Windows, go to Settings → Privacy & security → Microphone and turn on microphone access for desktop apps, then run `/voice` again. If your terminal isn't listed in the macOS settings, see [Terminal not listed in macOS Microphone settings](#).
- `No audio recording tool found` **on Linux**: the native audio module could not load and no fallback is installed. Install SoX with the command shown in the error message, for example `sudo`

```
apt-get install sox .
```

- **Voice mode could not find a working audio recorder in WSL** : WSLg routes audio through PulseAudio rather than an ALSA device, so SoX needs its PulseAudio backend installed explicitly. Run `sudo apt install sox libsox-fmt-pulse` . Installing `sox` alone pulls in the ALSA backend, which cannot record on WSL because there is no `/dev/snd` device.
- **Voice input is failing repeatedly and has been paused** : voice dictation hit several start-up failures in a row and stopped attempting new sessions until one succeeds. This usually means the microphone or audio stack on this host can't capture audio, for example a headless server, a remote shell with no audio passthrough, or a denied microphone permission. Confirm a working input device, fix the underlying cause from the entries above, then trigger voice again.
- **Nothing happens when holding Space in hold mode**: watch the prompt input while you hold. If spaces keep accumulating, voice dictation is likely off; run `/voice hold` to enable it. If only one or two spaces appear and then nothing, voice dictation is on but hold detection is not triggering. Hold detection requires your terminal to send key-repeat events, so it can't detect a held key if key-repeat is disabled at the OS level. Switch to tap mode with `/voice tap` to avoid the key-repeat requirement.
- **Tapping Space types a space instead of recording in tap mode**: the first tap only starts recording when the prompt input is empty. Clear the input first, or check that you are in tap mode by running `/voice tap` .
- **No audio detected from microphone** : recording started but captured silence. Confirm the correct input device is set as the system default and that its input level is not muted or near zero. On Windows, open Settings → System → Sound → Input and select your microphone. On macOS, open System Settings → Sound → Input.
- **No speech detected** : audio reached the transcription service but no words were recognized. Speak closer to the microphone, reduce background noise, and confirm your dictation language matches the language you are speaking.
- **Transcription is garbled or in the wrong language**: dictation defaults to English. If you are dictating in another language, set it in `/config` first. See [Change the dictation language](#).

Terminal not listed in macOS Microphone settings

If your terminal app does not appear under System Settings → Privacy & Security → Microphone, there is no toggle you can enable. Reset the permission state for your terminal so the next `/voice` run triggers a fresh macOS permission prompt.

Run `tccutil reset Microphone <bundle-id>`, replacing `<bundle-id>` with your terminal's identifier: `com.apple.Terminal` for the built-in Terminal, or `com.googlecode.itterm2` for iTerm2. For other terminals, look up the identifier with `osascript -e 'id of app "AppName"'`.

 You can run `tccutil reset Microphone` without a bundle ID, but it revokes microphone access from every app on your Mac, including apps like Zoom or Slack. Each app will need to re-request access on next use, so don't run it during an active call.

2 Quit and relaunch your terminal

macOS won't re-prompt a process that is already running. Quit the terminal app with Cmd+Q, not just close its windows, then open it again.

3 Trigger a fresh prompt

Start Claude Code and run `/voice`. macOS prompts for microphone access; allow it.

See also

- **Customize keyboard shortcuts**: rebind `voice:pushToTalk` and other CLI keyboard actions
- **Configure settings**: full reference for `voice`, `language`, and other settings keys
- **Interactive mode**: keyboard shortcuts, input modes, and session controls
- **Commands**: reference for `/voice`, `/config`, and all other commands